LEÇON 4 : Proof-theoretic semantics, vérificationnisme et pragmatisme

Éliminateurs à la main, propriétés de l'égalité, et `Compute`

Pablo Donato

07/04/2026

Nous avons vu aujourd'hui que les connecteurs logiques sont définis dans Rocq comme des **types inductifs**, dont les constructeurs sont exactement les règles d'introduction de la déduction naturelle. La tactique `destruct` génère automatiquement un `match ... with ... end` pour les éliminer. Dans ces exercices, vous écrirez ce `match` à la main pour voir ce qui se passe sous le capot. Vous décortiquerez ensuite les propriétés fondamentales de l'égalité, dont le comportement est lui aussi justifié par la sémantique vérificationniste.

```
Section Lesson4.
Variables A B C : Prop.
```

1 Éliminateurs à la main (`match ... with`)

Rappel : le type `A /\ B` a un seul constructeur `conj : A -> B -> A /\ B`. Pour utiliser une hypothèse `H : A /\ B`, on "pattern-matche" ou "filtre par motif" :

```
match H with
| conj a b => (* a : A, b : B disponibles ici *)
end
```

Le type `A \/ B` a deux constructeurs : `or_introl : A -> A \/ B` et `or_intror : B -> A \/ B`. Le filtrage génère deux branches :

```
match H with
| or_introl a => ...
| or_intror b => ...
end
```

Voici un exemple complet pour illustrer la démarche : on extrait les composantes avec `match`, puis on reconstruit avec `conj`.

```
(* Exemple : symetrie de la conjonction *)
Lemma and_sym : A /\ B -> B /\ A.
Proof.
  intro H.
  exact (match H with
         | conj a b => conj b a
         end).
Qed.
```

Exercice 1: Extraction de la deuxième composante

Sans utiliser `destruct`, prouvez que $A \wedge B \Rightarrow B$ en écrivant un `match` explicite.

```
Lemma and_elim_2 : A /\ B -> B.
Proof.
  (* A COMPLETER *)
Admitted.
```

Exercice 2: Symétrie de la disjonction

Sans utiliser `destruct`, prouvez $A \vee B \Rightarrow B \vee A$. Votre `match` doit avoir deux branches — l'une utilisant `or_intror`, l'autre `or_introl`.

```
Lemma or_sym : A \/ B -> B \/ A.
Proof.
  intro H.
  (* A COMPLETER *)
Admitted.
```

Exercice 3: De la conjonction vers la disjonction

Utilisez `match` pour extraire A de l'hypothèse $A \wedge B$, puis injectez-le dans $A \vee C$ avec `or_introl`.

```
Lemma and_to_or : A /\ B -> A \/ C.
Proof.
  (* A COMPLETER *)
Admitted.
```

Remarque 1: Correspondance avec `destruct`

À tout moment, vous pouvez utiliser `Print` pour voir le terme de preuve réel que Rocq a construit :

```
Print and_sym.
```

Vous constaterez que `destruct H as [a b]` génère exactement le même `match` que vous venez d'écrire à la main. La tactique n'est qu'un raccourci syntaxique.

2 Propriétés de l'égalité — λ-termes à la main

L'éliminateur de `eq` dérivé automatiquement par Rocq est `eq_ind` :

```
eq_ind : forall (A : Type) (x : A) (P : A -> Prop),
         P x -> forall y : A, x = y -> P y
```

C'est le **principe de Leibniz** : si $P(x)$ est vrai et $x = y$, alors $P(y)$ est vrai. La tactique `rewrite H` l'applique silencieusement.

Stratégie : pour prouver $Q(y)$ sachant `H : x = y`, choisir $P := \lambda z.\ Q(z)$ de sorte que $P(x) = Q(x)$, puis écrire `eq_ind x P (preuve de Q x) y H`.

L'exemple de la symétrie (vu dans `cours.v`) illustre la démarche :

```
(* Exemple : symetrie.  P := fun z => z = x, donc P x = (x = x). *)
```

```
Definition eq_sym_terme : forall (x y : T), x = y -> y = x :=
  fun x y H => eq_ind x (fun z => z = x) eq_refl y H.
```

```
Variable T : Type.
```

Exercice 4: Transitivité — λ-terme pur

Pour prouver $x = z$ sachant $H_1 : x = y$ et $H_2 : y = z$, choisir $P := \lambda w.\ x = w$. Alors $P(y) = (x = y)$, prouvé par H_1. Appliquer `eq_ind` à $H_2 : y = z$ pour obtenir $P(z) = (x = z)$.

```
Definition eq_trans : forall (x y z : T), x = y -> y = z -> x = z
    :=
  todo. (* A COMPLETER *)
```

Exercice 5: Congruence — λ-terme pur

Si $x = y$, alors $f(x) = f(y)$ pour toute $f : T \to T$. Choisir $P := \lambda z.\ f(x) = f(z)$, de sorte que $P(x) = (f(x) = f(x))$, prouvé par `eq_refl`.

```
Definition f_equal : forall (f : T -> T) (x y : T), x = y -> f x =
    f y :=
  todo. (* A COMPLETER *)
```

3 Expérimenter avec `Compute`

La commande `Compute` évalue une expression jusqu'à sa forme normale. Elle illustre l'**élimination des détours** (*cut elimination*) : un constructeur immédiatement suivi de son éliminateur se réduit en une étape.

```
Variable (a : A) (b : B) (f : A -> C) (g : B -> C).
```

Exécutez les commandes suivantes et observez les résultats :

```
(* Detour fonctionnel (beta) *)
Compute (fun x : A => x) a.

(* Detour conjonctif *)
Compute (match conj a b with conj x y => x end).

(* Detour disjonctif *)
Compute (match or_introl a : A \/ B with
         | or_introl x => f x
         | or_intror y => g y
         end).
```

Exercice 6: Observer la réduction

1. Que retourne chacune des trois commandes `Compute` ci-dessus ?
2. Que se passe-t-il si vous remplacez `or_introl a` par `or_intror b` dans le troisième exemple ?
3. *(Bonus)* Construisez un exemple de "double détour" — deux introductions suivies de deux éliminations — et vérifiez avec `Compute` que la normalisation donne

directement la valeur finale.

```
End Lesson4.
```